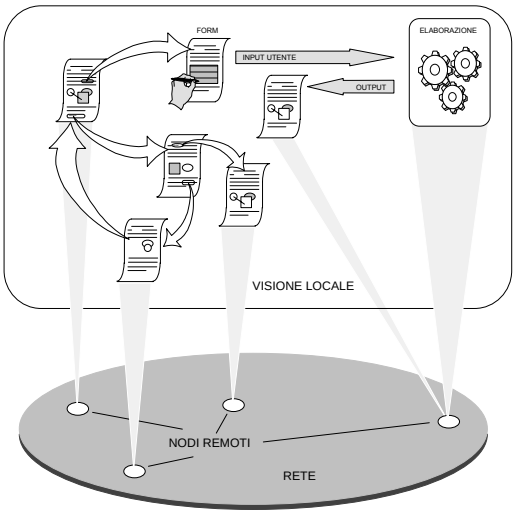
CdL in Ingegneria Gestione Industriale
Corso di Reti di Calcolatori

Il Web

Franco Zambonelli
A.A. 2005-2006

WWW (Mosaic, Netscape, Explorer)

strutturazione ipertestuale delle informazioni (trasparenza della allocazione delle informazioni) e uso di interfacce grafiche (semplicità di utilizzo)



Facendo "click"su una parola/immagine si "espande" una parte del documento che interessa. Non e' necessario "percepire" il fatto che il documento sia sul file system locale o su una macchina remota.

World Wide Web (WWW)

CERN (1989)
Progetto di integrazione in forma ipertestuale delle risorse esistenti in INTERNET

Scopi

- Trasparenza accesso e allocazione tramite nevigazione su ipertesti
- Presentazione multimediale
- Interfaccia unica per protocolli diversi (integrazione con gli altri protocolli)
- Modificabilità e condivisione delle informazioni

Ampia scelta di interfacce testuali e grafiche
Possibilità di estensioni sperimentali del sistema

Componenti

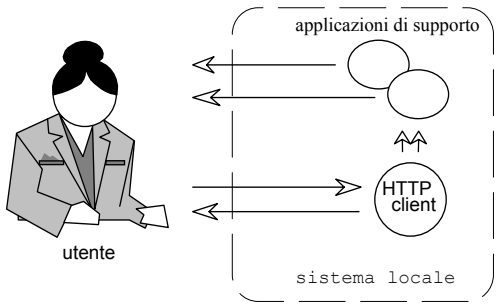
- Browser (presentazione e gestione richieste)
- Server (accesso e invio informazioni)
- Helper applications (particolari presentazioni)
- Applicazioni CGI (esecuzione remota)
- Applet (esecuzione locale)

Specifiche standard

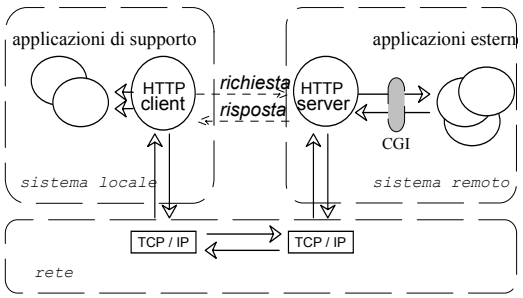
- Sistema di indirizzamento universale URI e URL (Uniform Resource Identifier/Location)
- Protocollo HTTP (HyperText Transfer Protocol)
- Linguaggio HTML (HyperText Markup Language)
- Interfaccia CGI (Common Gateway Interface)
- Linguaggio Java per Applet

SISTEMA **WWW**

Cliente e sua interazione



Il Cliente HTTP usa un modo cliente/servitore nei confronti di un server per volta e può anche interagire con risorse locali



URL Uniform Resource Locators

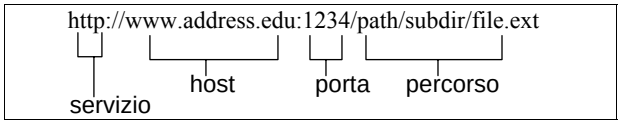
nomi unici per le risorse del sistema
specificati dal cliente per determinare il servitore

o **Uniform Resource Locators (URL):**

- nodo contenente la risorsa (*documento o dati*)
- protocollo di accesso alla risorsa (*e.g. http, gopher*)
- numero di porta TCP (*porta di default del servizio*)
- localizzazione della risorsa nel server.

```
<protocollo>[://<host>][:<porta>][<percorso>]
```

Sono riconosciuti i servizi internet e relativi protocolli =>
http, gopher, ftp, wais, telnet, news, nntp, e mail



Uso di default per localizzare risorse

Un **URL** può anche determinare un insieme di risorse:
ad esempio versioni multilingue tra cui scegliere

HTTP HyperText Transfer Protocol

protocollo di interfaccia tra cliente e servitore
Uso di TCP e di connessione (porta **80** default)

Caratteristiche HTTP:

- request/response
- one-shot connection
- stateless

Request/response: richiesta e ricezione di dati.
One-shot connection: la connessione TCP è mantenuta solo per il tempo necessario a trasmettere i dati
Stateless: non mantiene nessuna informazione tra una richiesta e la successiva

in genere:

- **richiesta** del cliente con **informazioni** per il **server**
- **risposta** con informazioni dal server

il cliente può determinare una forma di scelta (**negoziazione**) sulle informazioni ed i servizi

```
HTTP-message = Simple-Request          ;HTTP/0.9
   / Simple-Response
   / Full-Request                      ;HTTP/1.0
   / Full-Response
```

NON c'e' stato del server

Il Concetto di URL

L'indirizzo internet di un computer lo specifica univocamente nel mondo

Il nome di un file lo specifica univocamente nel suo file system

QUINDI:

Componendo indirizzo Internet e nome file si identifica univocamente un file a livello mondiale:

NOME COMPUTER/PERCORSO_E_NOME_DEL_FILE

pcfz.unimo.it/CorsoRE/Internet.doc

Ovviamente preceduto dal nome del protocollo che si deve (o si vuole) usare per accedere il file. Di default http.

http://pcfz.unimo.it/CorsoRE/Internet.doc

ftp://pcll.unibo.it/CorsoMO/Windows.doc

Windows98 integra questa visione del file system a livello globale in un ambiente in cui "navigare" nel proprio file system o "navigare" su Internet è la stessa cosa.

NON SOLO: accesso a risorse sul proprio computer (file identificati da un pathname)
MA ANCHE: accesso a risorse remote in Internet (file identificati da una URL)

Come per i file, è possibile collegare al desktop URL.

HTML HyperText Markup Language

HTML è un linguaggio di specifica delle informazioni che deriva da SGML (Standard Generalized Markup Language). E' un **markup language** (TeX, RTF).

I linguaggi markup usano dei **tag** definiti **funzionalmente** per caratterizzare graficamente il testo incluso.

tag HTML

testo di tipo header 1: `<H1>testo</H1>`
testo in grassetto: `<STRONG>testo</STRONG>` oppure `<B>testo</B>`
Visualizzazione dipendente dal browser

link: `<A HREF = "URLdestinazione"> descrizione </A>`

immagini: `<IMG SRC = "myimage.gif">`

applet Java:
`<APPLET CODE="Hello.class" WIDTH=100 HEIGHT=80>`

HTML **molto semplice** per non complicare il cliente

versione	browser	proprietà
1.0	storico	header, liste, enfasi
2.0	Mosaic	Inline Image, form
2.1	Netscape/Microsoft	tabelle, allineamento
3.2	Netscape/Microsoft	frame, ...
4.0	Netscape/Microsoft	Stili, JavaScript

Esempio pagina HTML (codice)

```
<HEAD>
<TITLE>Page Title</TITLE>
</HEAD>

<BODY>
<H1>Example of HTML document</H1>

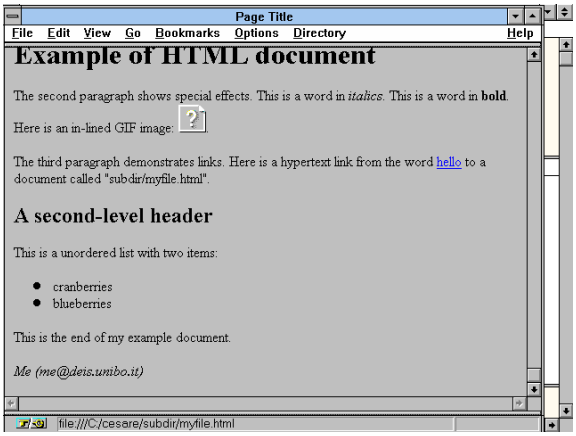
The second paragraph shows special effects.
This is a word in <I>italics</I>.  This is a
word in <B>bold</B>. Here is an in-lined GIF
image: <IMG SRC = "myimage.gif">. <P>

The third paragraph demonstrates links.  Here
is a hypertext link from the word <A HREF =
"subdir/myfile.html">hello</A>  to  a  document
called "subdir/myfile.html". <P>

<H2>A second-level header</H2>

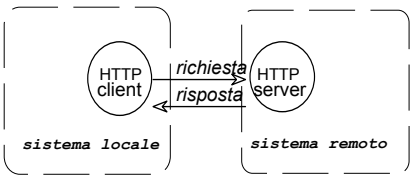
This is a unordered list with two items: <P>
<UL>
<LI> cranberries
<LI> blueberries
</UL>
This is the end of my example document. <P>
<ADDRESS> Me (me@deis.unibo.it) </ADDRESS>
</BODY>
```


Esempio pagina HTML (visualizzazione)



Per approfondire la conoscenza di HTML consultare il sito Web del corso.

Programmazione client/server in WWW

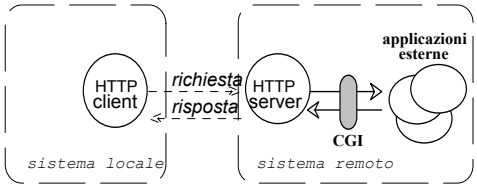


Possibilità di avere ***risposta*** con informazioni dinamiche

Che tipo di elaborazione delle informazioni e
dove viene eseguita

richiesta	risposta	tipo di elaborazione
Documento HTML	Statica (la pagina è un file, non modificabile)	semplice trasferimento file dal server
CGI	dinamica	qualunque elaborazione sul nodo server
Java applet	statica	codice dal server non modificabile, esecuzione sul client
Java applicazione	dinamica	server elabora dinamicamente il codice (in base alla richiesta), esecuzione dinamica sul client

Common Gateway Interface (CGI)



CGI è uno **standard** per interfacciare un server WWW con applicazioni esterne (residenti sulla macchina server)

CGI fornisce all'utente la capacità di eseguire una applicazione sulla macchina server remota

richiesta	*risposta*	*tipo di elaborazione*
CGI	dinamica	Qualunque elaborazione, **sul nodo server**

La risposta ottenuta dal server è "dinamica", in quanto risultante dalla esecuzione di un programma sul server.

Programmazione CGI

Una applicazione CGI permette agli utenti di eseguire una applicazione sul nodo dove risiede il server www.

Applicazioni CGI possono essere scritte in: C/C++, Fortran, PERL, TCL, Any Unix shell, Visual Basic ….etc etc

Normale attivazione di una CGI:

- Si invia al server un messaggio (ciò avviene tipicamente riempiendo moduli, i cui dati serviranno come input al programma)
- Il messaggio scatena l'esecuzione del programma CGI
- Il programma CGI genera come output una pagina HTML in cui inserisce i risultati della sua esecuzione

Interfaccia tra server WWW e applicazione CGI

- **Variabili di ambiente** (non le vediamo in dettaglio)
- **Linea di comando**
- **Standard input**: il server ridirige sull'ingresso della applicazione CGI i dati ricevuti dal client (browser). Il numero di byte è nella varabile di ambiente CONTENT_LENGTH.
- **Standard output**: l'applicazione CGI manda il risultato dell-elaborazione sullo standard output (in formato HTML), verso il server, il quale prende i dati e li manda al client.

Client HTTP → server HTTP → CGI

Tipicamente, uso di **form**

```
<TITLE>Esempio di Form </TITLE>
<H1>Esempio di Form </H1>

<FORM METHOD="POST" ACTION="http://www-
lia.deis.unibo.it/cgi-bin/post-query">

Inserisci del testo: <INPUT NAME="entry">

e premi per invio: <INPUT TYPE="submit"
                    VALUE="Invio">
</FORM>
```

Visualizzazione **form**



Client HTTP → server HTTP → CGI

Attributi del **form tag**

```
<TITLE>Esempio di Form </TITLE>
<H1>Esempio di Form </H1>

<FORM METHOD="POST" ACTION="http://www-
lia.deis.unibo.it/cgi-bin/post-query">

Inserisci del testo: <INPUT NAME="entry">

e premi per invio: <INPUT TYPE="submit"
                    VALUE="Invio">
</FORM>
```

Dove:

ACTION URL di chi processa la query

METHOD metodo usato per sottomettere il form:

- **POST** il form con i dati è spedito come data body (metodo consigliato)
- **GET** il form con i dati è spedito attaccato all'URL (`action?name=value&name=value`)

caso `GET`

```
http://www-lia.deis.unibo.it
                /cgi-bin/get-query?entry=testo
```

caso `POST`

```
http://www-lia.deis.unibo.it
                /cgi-bin/post-query
```

e come data body:

```
                entry=testo
```


Applicazione CGI → server HTTP

Applicazione CGI usa **standard output** per mandare al server i dati. I dati sono identificati da un header.

Tipi di dati forniti:

- full document con il corrispondente MIME type (text/html, text/plain per testo ASCII, etc.)

  Esempio: per spedire una pagina HTML

  ```
  Content-type: text/html

  <HTML><HEAD>
  <TITLE>output di HTML da script CGI</TITLE>
  </HEAD><BODY>
  <H1>titolo</H1>
  semplice <STRONG>prova</STRONG>
  </BODY></HTML>
  ```

- reference a un altro documento

  Esempio: riferisco un documento gopher

  ```
  Content-type: text/html
  Location: gopher://httprules.foobar.org/0

  <HTML><HEAD>
  <TITLE > Sorry ... it moved </TITLE>
  </HEAD><BODY>
  <H1>go to gopher </H1>
  Now available at
  <A HREF="gopher://httprules.foobar.org/0">
            new location</A> of gopher server.
  </BODY></HTML>
  ```

Applicazione CGI

Esempio: generazione della pagina di risposta (caso full document)

```
#include <stdio.h>
...............

main(int argc, char *argv[]) {
  int cl;
```

generazione di un full document in risposta

```
  printf("Content-type: text/html");

  cl = atoi(getenv("CONTENT_LENGTH"));

  for(x=0;cl && (!feof(stdin));x++) {
    ...
```

elaborazione dell'input (stdin)

```
    ...
  }

  printf("<H1>Query Results</H1>");
  printf("You submitted ...");

  for(x=0; x <= m; x++)
    printf("...........", ... , ....);
}
```


PROBLEMI introdotti

Problema del **traffico**
- alcuni formati di informazioni possono richiedere un **eccesso di banda**
- *necessità di adeguare le infrastrutture*

Problema della **sicurezza**
- la trasparenza richiede la integrazione con i sistemi locali di esecuzione/visualizzazione: intrusioni o virus da controllare
- riservatezza delle transazioni

Problema della **visualizzazione**
- alcuni formati richiedono una lunga visualizzazione: tempi di accesso molto rallentati

sfruttare la possibile concorrenza/asincronismo

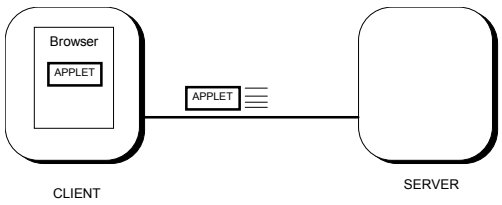
Uso di clienti con strategie adatte a
- abbreviare il **tempo di risposta**
- favorire il browsing delle informazioni
  - **(web navigation e ricerche mediante robot, spider, worm, ecc.)**

evoluzione dei protocolli

per garantire una apertura alle esigenze man mano determinate

Applet

- una applet (piccola applicazione) viene eseguita all'interno di un browser WWW come Explorer o Netscape Navigator
- tramite il tag **<applet>** è possibile dire al browser di scaricare il codice (il file .class) e di eseguirlo
- al contrario del CGI, la computazione viene eseguita sul client



richiesta	risposta	tipo di elaborazione
Java applet	statica	codice dal server non modificabile, esecuzione sul client

Applet Java

Stessa semantica del recupero di una immagine:
- vedere una applet ⇨ eseguirla

Classe
`java.applet.Applet`

Metodi
- per interagire con l'ambiente in cui viene eseguita
- per caricare e gestire immagini e suoni
- per ottenere parametri contenuti nel file HTML
- per ottenere informazioni sul proprio stato (attiva/disattiva, dimensioni, ...)

Sicurezza
- l'applet viene eseguita in una "sandbox" per cui può accedere solo alle risorse dell'host da cui viene prelevata, in particolare:
  - ⇒ connessioni di rete: solo con host di provenienza
  - ⇒ non può scrivere sul file system locale al browser dove viene eseguita
- nel jkd 1.2 maggiore flessibilità
  - ⇒ possibilità di specificare quali operazioni può effettuare un'applet a seconda della provenienza e/o firma digitale

Esempio di applet

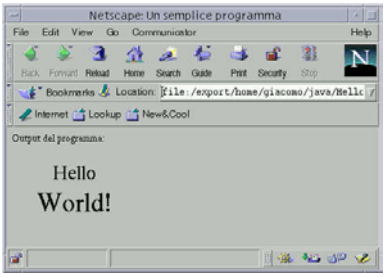
File di codice HelloWorld.java

```
import java.applet.Applet;
import java.awt.Graphics;
import java.awt.Font;

public class HelloWorld extends Applet
{  public void paint(Graphics g)
  {   g.setFont(new Font("TimesRoman", Font.PLAIN, 30));
      g.drawString("Hello", 50, 30);
      g.setFont(new Font("TimesRoman", Font.PLAIN, 40));
      g.drawString("World!", 30, 70);
} }
```

File HelloWorld.html per il browser

```
<HTML> <HEAD>
<TITLE> Un semplice programma </TITLE>
</HEAD> <BODY>
Output del programma:<BR>
<APPLET CODE="HelloWorld.class" WIDTH=250 HEIGHT=100>
</APPLET>
    </BODY> </HTML>
```



Java Script

In alternativa alle Applet, si possono rendere le pagine HTML dinamiche grazie all'uso di JavaScript.

Uno speciale TAG HTML può contenere dei piccoli pezzi di programma scritti in un linguaggio di programmazione che è una versione semplificata di Java.

I programmi specificano quando devono essere eseguiti. In particolare, specificano gli "eventi" che ne causano l'esecuzione: il click del mouse in un certo punto, il passaggio del mouse su una certa parola.

L'esecuzione può causare azioni diverse, quali l'apparizione di finestre o il cambiamento dinamico dell'aspetto della pagine.

Esempio:

```
<a HREF="provejs.html"
onClick="
var n = 0;
alert('apro 2 finestre');
n = n + 1;
alert('Apro finestra numero ' + n);
window.open('http://www.dsi.unimo.it');
n = n + 1;
alert('Apro finestra numero ' + n);
window.open('http://www.dsi.unimo.it')
">
DSI</a><br>
```

Guardare il sito Web del corso per approfondire le conoscenze su JavaScript

La ricerca di informazioni sul WEB

Il web è un ipertesto (un grafo) con milioni di nodi → problema di reperire le informazioni utili

Esistono **indici del web**, ottenuti usando dei programmi che esplorano la rete raccogliendo le informazioni in un database.

Esempio:
AltaVista (http://altavista.digital.com) Web index:

- più di 100 milioni pagine da più di 500.000 servers
- 10 milioni articoli da 14,000 Usenet news groups.

E' acceduto più di 50 milioni di volte al giorno

I motori di ricerca

Estraggono parole chiave da ogni pagina web.
Creano basi di dati indicizzate sulla base delle parole chiave che vengono consultati quando l'utente digita una serie di parole chiave (il tutto avviene con delle CGI).

problemi:

- titoli pagine spesso poco significativi
- analisi intere pagine costosa
- pagine solo video o audio, oppure active map

Quindi, spesso non sono utilissimi….